

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Section 1: Regular Expression Pattern Generation

Student Details:**Name:** John Doe**Email:** john.doe123@gmail.com**Mobile:** +91-9876543210**Password:** P@ssw0rd123**Date of Birth:** 15/08/2004**Register Number:** 23AIML1056**Department:** Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:**Email ID**

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Section 1: Regular Expression Pattern Generation

Explanation

This program validates the student's Email ID, Mobile Number, Strong Password, Date of Birth, and Register Number using regular expressions.

Code

```
import re
email = "john.doe123@gmail.com"
mobile = "+91-9876543210"
password = "P@ssw0rd123"
dob = "15/08/2004"
reg_no = "23AIML1056"
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
mobile_pattern = r'^\+91-[6-9]\d{9}$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$$%&!])[A-Za-z\d@#$$%&!]{8,}$'
dob_pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/d{4}$'
reg_pattern = r'^\d{2}[A-Z]{4}\d{4}$'
print("Email:", "Valid" if re.fullmatch(email_pattern, email) else "Invalid")
print("Mobile:", "Valid" if re.fullmatch(mobile_pattern, mobile) else "Invalid")
print("Password:", "Valid" if re.fullmatch(password_pattern, password) else "Invalid")
print("Date of Birth:", "Valid" if re.fullmatch(dob_pattern, dob) else "Invalid")
print("Register Number:", "Valid" if re.fullmatch(reg_pattern, reg_no) else "Invalid")
```

Step-by-Step Explanation

1. Import the `re` module.
2. Store the student details in variables.
3. Create separate regular expression patterns for Email, Mobile Number, Password, Date of Birth, and Register Number.
4. Use `re.fullmatch()` to compare each value with its pattern.
5. Print whether each detail is **Valid** or **Invalid**.

Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/NLP ASSESSMENT/AST-1.py =====
Email: Valid
Mobile: Valid
Password: Valid
Date of Birth: Valid
Register Number: Valid
'
```

Register Number Regex Explanation

`^\d{2}[A-Z]{4}\d{4}$`

- `^` → Start of the string
- `\d{2}` → Exactly 2 digits (Year: 23)
- `[A-Z]{4}` → Exactly 4 uppercase letters (AIML)
- `\d{4}` → Exactly 4 digits (1056)
- `$` → End of the string

This matches:

```
23AIML1056
24CSEA1234
22ECEEE0456
```

and rejects invalid formats.

Section 2: Search for a String

Explanation

This program searches for the word "AI" in the given passage using Python's `re` module. It displays the first occurrence, its starting and ending positions, counts the total occurrences, and prints a message if the word is not found.

Code

```
import re
text = """Artificial Intelligence (AI) is transforming industries across the world.
AI is used in healthcare to assist doctors in diagnosis, in banking to detect
fraud,
and in education to provide personalized learning experiences.
Many companies invest heavily in AI research because AI improves efficiency
and enables intelligent decision-making. As AI continues to evolve,
professionals with AI skills are in high demand."""
pattern = r"\bAI\b"
match = re.search(pattern, text)
if match:
    print("First Occurrence :", match.group())
    print("Starting Position :", match.start())
    print("Ending Position :", match.end())

    count = len(re.findall(pattern, text))
    print("Total Occurrences :", count)
else:
    print("Word 'AI' not found.")
```

Step-by-Step Explanation

1. Import the `re` module.
2. Store the given passage in a string variable.
3. Define the regex pattern `\bAI\b` to search for the whole word **AI**.
4. Use `re.search()` to find the first occurrence.
5. Display the matched word using `group()`.
6. Display the starting position using `start()`.
7. Display the ending position using `end()`.
8. Use `re.findall()` to count the total occurrences.
9. If the word is not found, display an appropriate message.

Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/NLP ASSESSMENT/AST-2.py =====
First Occurrence : AI
Starting Position : 25
Ending Position : 27
Total Occurrences : 6
'
```

Regex Used

`\bAI\b`

Meaning

- `\b` → Word boundary
- `AI` → Matches the word **AI**
- `\b` → Ensures only the complete word is matched (not words like **AIML** or **PAINT**)

Section 3: Split the Documentation into Sentences and Words

Explanation

This program uses the `re.split()` method to split the given passage into sentences and words. It also counts the total number of sentences and words.

Code

```
import re
text = """Artificial Intelligence (AI) is transforming industries across the world.
AI is used in healthcare to assist doctors in diagnosis, in banking to detect
fraud, and in education to provide personalized learning experiences. Many
companies invest heavily in AI research because AI improves efficiency and enables
intelligent decision-making. As AI continues to evolve, professionals with AI
skills are in high demand."""
sentences = re.split(r'[.!?]+', text)
sentences = [s.strip() for s in sentences if s.strip()]
words = re.split(r'\s+', text)
print("Total Sentences:", len(sentences))
print("Total Words:", len(words))
print("\nSentences:")
for sentence in sentences:
    print(sentence)
print("\nWords:")
print(words)
```

Step-by-Step Explanation

1. Import the `re` module.
2. Store the given passage in a string variable.
3. Use `re.split(r'[.!?]+', text)` to split the passage into sentences.
4. Remove any empty strings using a list comprehension.
5. Use `re.split(r'\s+', text)` to split the passage into words.
6. Count and display the total number of sentences and words.
7. Display the extracted sentences and words.

Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/NLP ASSESSMENT/AST-3.py =====
Total Sentences: 4
Total Words: 60

Sentences:
Artificial Intelligence (AI) is transforming industries across the world
AI is used in healthcare to assist doctors in diagnosis, in banking to detect fr
aud, and in education to provide personalized learning experiences
Many companies invest heavily in AI research because AI improves efficiency and
enables intelligent decision-making
As AI continues to evolve, professionals with AI skills are in high demand

Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'acro
ss', 'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'd
octors', 'in', 'diagnosis,', 'in', 'banking', 'to', 'detect', 'fraud,', 'and', '
in', 'education', 'to', 'provide', 'personalized', 'learning', 'experiences.', '
Many', 'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI'
, 'improves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making.',
'As', 'AI', 'continues', 'to', 'evolve,', 'professionals', 'with', 'AI', 'skill
s', 'are', 'in', 'high', 'demand.']
```

Regex Used
Split Sentences
[.!?]+

Meaning:

- [] → Character set
- . → Full stop
- ! → Exclamation mark
- ? → Question mark
- + → One or more punctuation marks

Split Words
\s+

Meaning:

- \s → Matches any whitespace (space, tab, newline)
- + → Matches one or more whitespace characters

Section 4: Email and Strong Password Validation

Explanation

This program validates an Email ID and a Strong Password using Python's `re` module. It checks whether the inputs satisfy the given validation criteria.

Code

```
import re
email = input("Enter Email ID: ")
password = input("Enter Password: ")
email_pattern = r'^[a-zA-Z0-9]+[a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.(com|org|edu|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*]) [^\s]{8,}$'
if re.fullmatch(email_pattern, email):
    print("Valid Email ID")
else:
    print("Invalid Email ID")
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
```

Step-by-Step Explanation

1. Import the `re` module.
2. Accept the Email ID and Password from the user.
3. Store the regular expression patterns for email and password.
4. Use `re.fullmatch()` to validate the Email ID.
5. Use `re.fullmatch()` to validate the Password.
6. Display whether the Email ID is valid and whether the Password is strong or weak.

Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/NLP ASSESSMENT/AST-4.py =====
Enter Email ID: cryo2507@gmail.com
Enter Password: qwerty5526n
Valid Email ID
Weak Password
```

Regex Used

Email Pattern

`^[a-zA-Z0-9]+[a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.(com|org|edu|in)$`

Meaning:

- `^[a-zA-Z0-9]+` → Starts with one or more letters or digits.
- `[a-zA-Z0-9._%+-]*` → May contain letters, digits, `.`, `_`, `%`, `+`, or `-`.
- `@` → Must contain the `@` symbol.
- `[a-zA-Z0-9.-]+` → Valid domain name.
- `\.(com|org|edu|in)` → Valid domain extension.
- `$` → End of the string.

Password Pattern

`^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*]) [^\s]{8,}$`

Meaning:

- $(?=.*[A-Z]) \rightarrow$ At least one uppercase letter.
- $(?=.*[a-z]) \rightarrow$ At least one lowercase letter.
- $(?=.*\d) \rightarrow$ At least one digit.
- $(?=.*[!@#$%^&*]) \rightarrow$ At least one special character.
- $[\w]{8,} \rightarrow$ Minimum 8 characters with no whitespace.
- $\$ \rightarrow$ End of the string.

Section 5: Compile Once, Validate Many

Explanation

This program uses the `re.compile()` method to compile the email validation pattern only once. The compiled pattern is then reused to validate multiple email IDs efficiently.

Code

```
import re
email_pattern = re.compile(
    r'^[a-zA-Z0-9]+[a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.(com|org|edu|in)$'
)
emails = [
    "john@gmail.com",
    "student123@yahoo.org",
    "abc@college.edu",
    "test_user@domain.in",
    "invalid@email",
    "@gmail.com"
]
for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid Email")
    else:
        print(email, "-> Invalid Email")
```

Step-by-Step Explanation

1. Import the `re` module.
2. Compile the email validation pattern using `re.compile()`.
3. Store multiple email IDs in a list.
4. Use the compiled pattern with `fullmatch()` to validate each email ID.
5. Display whether each email ID is valid or invalid.

Output

```
===== RESTART: C:\Users\IDRKorIDRC\Desktop\NLP ASSESSMENT\AST-5.py =====
john@gmail.com -> Valid Email
student123@yahoo.org -> Valid Email
abc@college.edu -> Valid Email
test_user@domain.in -> Valid Email
invalid@email -> Invalid Email
@gmail.com -> Invalid Email
```

Regex Used

`^[a-zA-Z0-9]+[a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.(com|org|edu|in)$`

Meaning

- `^` → Start of the string.
- `[a-zA-Z0-9]+` → Starts with one or more letters or digits.
- `[a-zA-Z0-9._%+-]*` → Allows letters, digits, `.`, `_`, `%`, `+`, and `-`.
- `@` → Must contain the `@` symbol.
- `[a-zA-Z0-9.-]+` → Matches the domain name.
- `\.(com|org|edu|in)` → Matches valid domain extensions.
- `$` → End of the string.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____